

P2617R0: Responses to NB comments on DTS 12907 "Extensions to C++ for Transactional Memory Version 2"

Introduction

SC22 N5721 contains the six collated NB comments resulting from the DTS ballot on C++ Extensions for Minimal Transactional Memory. This paper presents the proposed responses to the NB comments, presented as changes against WG21 N4906 "Working Draft, Extensions to C++ for Transactional Memory Version 2", the base document for the DTS ballot.

Wording changes

CA-001: Transferring control into an atomic block

Disposition: Accept.

Change in subclause 8.8 [stmt.tx] paragraph 3:

~~A goto or switch statement shall not be used to transfer control into an atomic block.~~ **A case or default label appearing within an atomic block shall be associated with a switch statement (8.5.3 [stmt.switch]) within the same atomic block. A label (8.2 [stmt.label]) declared in an atomic block shall only be referred to by a statement in the same atomic block.**

CA-002: Exiting an atomic block via return or goto

Disposition: Accept with modifications.

Change in 8.8 [stmt.tx] paragraph 2:

The start of the atomic block is immediately ~~before~~ **after** the opening { of the *compound-statement*. ~~The~~ **Evaluation of the** end of the atomic block ~~is immediately after the closing }~~ **occurs when the invocation of the destructor for a hypothetical automatic variable of class type declared at the start of the atomic block would happen (8.7.1 [stmt.jump.general]).**

CA-003: Cross-reference to manifest-constant evaluated

Disposition: Accept.

Change in 8.8 [stmt.tx] paragraph 6:

If the execution of an atomic block evaluates any of the following outside of a manifestly constant-evaluated context (**7.7 [expr.const]**), the behavior is implementation-defined: ...

CA-004: Implementation-defined behavior for library functions

Disposition: Accept.

Change in 8.8 [stmt.tx] paragraph 6:

- ...
- an invocation of a function ~~other than one of the standard library functions specified in 16.4.6.17, unless the function is inline with a reachable definition; , unless~~
 - **the function is inline with a reachable definition or**
 - **the function is a library function that may be used in an atomic block (16.4.6.17);**
- ...

CA-005: Setting the global locale in an atomic block

Disposition: Accept.

Add a new bullet in 16.4.6.17 [atomic.use]:

- ...
- locale construction (28.3.1.3)
- **locale::global (28.3.1.6 [locale.statics])**
- ...

CA-006: Scope

Disposition: Accept.

Change in Clause 1 [scope] paragraph 1:

This document describes requirements for implementations of an interface that computer programs written in the C++ programming language ~~may~~ **can** use to ~~invoke algorithms with concurrent execution. The algorithms described by this document are realizable across a broad class of computer architectures.~~ **express groups of operations (known as transactions) that appear to execute atomically in relation to other transactions (concurrent or otherwise).**

Additional change: Initialization of variables with thread storage duration

Change in 8.8 [stmt.tx] paragraph 6:

- ...
- dynamic initialization of a variable with thread storage duration. **[Note: That includes the case when such an initialization is evaluated within an atomic block because the initialization was deferred (6.9.3.3 [basic.start.dynamic]). -- end note]**